

第 29 讲核心概念速查卡

PyTorch Autograd

一句话：Autograd = Forward 时记录计算依赖，Backward 时自动沿图执行 Chain Rule，把参数对 Loss 的梯度累积到 `.grad`。

1. 最小模板

```
x = torch.tensor(3.0, requires_grad=True)
loss = x ** 2
loss.backward()
print(x.grad)  # 6
```

2. 四个关键属性/动作

<code>requires_grad</code>	是否要求跟踪该 Tensor 对最终目标的梯度依赖
<code>is_leaf</code>	是否通常是用户/Parameter 直接创建的叶子节点
<code>grad_fn</code>	non-leaf Tensor 由哪种可微操作产生
<code>.grad</code>	backward 后累积得到的梯度，参数上最常查看

3. Backward 的底层

逆序遍历计算图 + 局部导数 + Chain Rule + 分支累加

4. 梯度默认累加

```
loss.backward()  # grad += new_grad
loss.backward()  # 若重新建图且不清零，会继续累加
```

训练中通常：

```
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

5. Linear 的梯度 Shape

若：

$$Y = XW^T + b$$

则：

$$dW = dY^T X, \quad db = \sum dY, \quad dX = dYW$$

并且：

$$dW.shape = W.shape, \quad db.shape = b.shape, \quad dX.shape = X.shape$$

6. no_grad vs detach

- `torch.no_grad()`：一段代码期间停止记录；常用于推理/参数手工更新。
- `tensor.detach()`：得到与原计算图断开的 Tensor；常用于记录/缓存/明确 stop-gradient。

7. 必记边界

`backward()` 只计算梯度，不负责参数更新。Autograd 也只会对“你实际写出的 Forward”正确求导；Forward 如果写错，梯度会对错误逻辑求得很正确。

8. 与 LLM 的关系

CrossEntropy Loss $\xrightarrow{\text{Autograd}}$ *Transformer Blocks* $\rightarrow$ *Millions/Billions of parameter.grad*

LoRA、Adapter、全参微调的一个核心区别，就是“哪些参数 `requires_grad=True` 并收到梯度”。